

JAVA DAY 5-15 TRAINING PACK

Classes, Objects, OOP, Exceptions, File Handling and Collections | Ashwini K

COURSE ROADMAP

Day 5: Classes & Objects
Day 6: OOP - Encapsulation & Inheritance
Day 7: OOP - Polymorphism, Abstraction & Interfaces
Day 8: Exception Handling
Day 9: File Handling
Day 10: Collections + List
Day 11: Queue, Deque, Stack, PriorityQueue
Day 12: Set
Day 13: Map
Day 14: Iterator, Comparable, Comparator
Day 15: Advanced Collections + Senior Interview Practice

HOW TO USE THESE NOTES

1. Read the concept box.
2. Trace the flow arrow.
3. Type and run every program yourself.
4. Complete the assignment without copying.
5. Explain the scenario answer aloud as an interview response.
6. Refactor one solution after it works.

Trainer: Ashwini K | Java / QA / SDET Training

DAY 5 - Classes and Objects

Ashwini K | Goal: Understand how Java models real-world entities using classes, objects, fields, constructors and methods.

CORE IDEA

Understand how Java models real-world entities using classes, objects, fields, constructors and methods.

Class	new	Object	Methods / Data	Result
-------	-----	--------	----------------	--------

Class -> new -> Object -> Methods / Data -> Result

Concept	Explanation
Class	Blueprint that defines data and behavior. Example: LoginPage, Customer, Payment.
Object	Runtime instance of a class created using new.
Instance variable	Non-static field; each object keeps its own value.
Constructor	Special member used to initialize an object; same name as class and no return type.
this	Refers to the current object; useful when parameter and field names are same.
new	Allocates an object and invokes its constructor.

Basic class and object

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println(name + " - " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Rahul";
        s.age = 25;
        s.display();
    }
}
```

Constructor + this keyword

```
class LoginUser {
    String username;
    String role;

    LoginUser(String username, String role) {
        this.username = username;
        this.role = role;
    }

    void printUser() {
        System.out.println(username + " -> " + role);
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        LoginUser u = new LoginUser("ashwini", "QA");
        u.printUser();
    }
}

```

QA-style TestResult object

```

class TestResult {
    String testName;
    String status;
    long durationMs;

    TestResult(String testName, String status, long durationMs) {
        this.testName = testName;
        this.status = status;
        this.durationMs = durationMs;
    }

    boolean isSlow(long threshold) {
        return durationMs > threshold;
    }
}

public class Main {
    public static void main(String[] args) {
        TestResult r = new TestResult("Login", "PASS", 1800);
        System.out.println(r.status);
        System.out.println("Slow test? " + r.isSlow(1500));
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create a Customer class with id, name, email and balance. Add constructor + display method.
2. Create a BrowserSession class with browser, version and environment. Add a method that prints a session key.
3. Build a TestCase class with testId, title, priority and status. Add a method that returns whether execution is allowed.
4. Create two objects with different data and explain why instance variables are not shared between them.

SCENARIO-BASED INTERVIEW PRACTICE

1. Your framework creates 500 test objects during a suite. What data should remain instance-level and what data could be static?
2. A developer says constructors can return values. How would you correct the statement and explain why?
3. You see duplicated setup code in 20 test classes. How can classes and objects reduce this duplication?

INTERVIEW QUICK REVISION

- Class = blueprint; object = instance.
- Constructor has class name and no return type.
- this refers to current object.
- Instance fields belong to each object.
- new creates object and triggers constructor.

DAY 6 - OOP - Encapsulation and Inheritance

Ashwini K | Goal: Apply data hiding, reusable parent behavior and controlled access in automation-oriented code.

CORE IDEA

Apply data hiding, reusable parent behavior and controlled access in automation-oriented code.

Private Data	Getter / Setter	Validation	Safe Object State
--------------	-----------------	------------	-------------------

Private Data -> Getter / Setter -> Validation -> Safe Object State

Concept	Explanation
Encapsulation	Bundle data and methods together and protect internal state using access modifiers.
private	Restricts direct access to the declaring class.
Getter / Setter	Controlled read/write access; setters can validate input.
Inheritance	Child class reuses or extends parent class behavior using extends.
super	Refers to parent class members or constructor.
IS-A relationship	Inheritance models a true specialization relationship.

Encapsulation with validation

```
class User {
    private String email;

    public void setEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.email = email;
    }

    public String getEmail() {
        return email;
    }
}

public class Main {
    public static void main(String[] args) {
        User u = new User();
        u.setEmail("qa@example.com");
        System.out.println(u.getEmail());
    }
}
```

Inheritance for automation pages

```
class BasePage {
    void click(String locator) {
        System.out.println("Click: " + locator);
    }
}

class LoginPage extends BasePage {
    void login(String user, String pwd) {
        click("#username");
    }
}
```

```

        click("#password");
        System.out.println("Login as " + user);
    }
}

public class Main {
    public static void main(String[] args) {
        LoginPage page = new LoginPage();
        page.login("ashwini", "secret");
    }
}

```

super constructor

```

class TestEnvironment {
    String env;

    TestEnvironment(String env) {
        this.env = env;
    }
}

class ApiEnvironment extends TestEnvironment {
    int timeout;

    ApiEnvironment(String env, int timeout) {
        super(env);
        this.timeout = timeout;
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create an Account class with private balance. Allow deposits and withdrawals only when business rules are satisfied.
2. Create BasePage -> LoginPage -> AdminLoginPage and identify which methods are inherited.
3. Create BaseTest with common setup values and a WebTest child that adds browser-specific data.
4. Add validation to setters for username, timeout and environment name.

SCENARIO-BASED INTERVIEW PRACTICE

1. A test class directly changes page object fields. How would you redesign it using encapsulation?
2. Your framework has BasePage, BaseTest and BaseApi classes. Which inheritance relationships are valid and which may indicate poor design?
3. A child class needs a parent constructor value. Explain the role of super() and constructor order.

INTERVIEW QUICK REVISION

- Encapsulation = data + behavior + controlled access.
- private is commonly used to protect fields.
- Inheritance uses extends.
- super accesses parent constructor/members.
- Prefer inheritance for stable IS-A relationships, not for every reuse need.

DAY 7 - OOP - Polymorphism, Abstraction and Interfaces

Ashwini K | Goal: Build flexible automation components using overriding, overloading, abstract classes and interfaces.

CORE IDEA

Build flexible automation components using overriding, overloading, abstract classes and interfaces.

Parent / Interface Reference	Actual Object	Runtime Selection	Behavior
------------------------------	---------------	-------------------	----------

Parent / Interface Reference -> Actual Object -> Runtime Selection -> Behavior

Concept	Explanation
Polymorphism	One reference can work with different object implementations.
Overloading	Same method name with different parameter lists; compile-time selection.
Overriding	Child provides its own implementation of an inherited method; runtime selection.
Abstract class	Can contain common implementation plus abstract methods; cannot be instantiated.
Interface	Defines a contract that implementing classes must follow.
Dynamic dispatch	Overridden method selected at runtime based on actual object type.

Method overriding

```
class Report {
    void generate() {
        System.out.println("Generic report");
    }
}

class ExtentReport extends Report {
    @Override
    void generate() {
        System.out.println("Generate Extent report");
    }
}

public class Main {
    public static void main(String[] args) {
        Report r = new ExtentReport();
        r.generate();
    }
}
```

Interface for notification

```
interface Notifier {
    void send(String message);
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("EMAIL: " + message);
    }
}
```

```

    }
}

class SlackNotifier implements Notifier {
    public void send(String message) {
        System.out.println("SLACK: " + message);
    }
}

public class Main {
    static void notifyUser(Notifier n) {
        n.send("Build completed");
    }

    public static void main(String[] args) {
        notifyUser(new EmailNotifier());
        notifyUser(new SlackNotifier());
    }
}

```

Overloading utility

```

class RetryUtil {
    void retry(String testName) {
        retry(testName, 1);
    }

    void retry(String testName, int count) {
        System.out.println("Retry " + testName + " " + count + " time(s)");
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Design a PaymentGateway interface with pay() and refund(). Create CardPayment and UpiPayment implementations.
2. Create an abstract TestReporter with write() and common header logic. Implement HtmlReporter and JsonReporter.
3. Create a SearchStrategy interface for WebSearch and ApiSearch. Use one service method to execute either strategy.
4. Write examples showing overloading vs overriding and annotate why the binding time differs.

SCENARIO-BASED INTERVIEW PRACTICE

1. Your framework must support Selenium, Playwright and API clients behind one test abstraction. Which OOP feature helps and why?
2. A manager class has 12 if/else blocks for notification type. How could an interface-based design reduce that branching?
3. An interviewer asks whether a static method can be overridden. Explain the difference between hiding and overriding.

INTERVIEW QUICK REVISION

- Overloading -> compile time; overriding -> runtime.
- Reference type controls accessible members; actual object controls overridden method at runtime.
- Abstract classes are for shared base behavior plus required methods.
- Interfaces define contracts and support multiple implementations.
- Use polymorphism to reduce large conditional branches.

DAY 8 - Exception Handling

Ashwini K | Goal: Handle predictable failures without hiding defects; design useful exceptions for automation and application flows.

CORE IDEA

Handle predictable failures without hiding defects; design useful exceptions for automation and application flows.

Risky Step	try	Exception?	catch / propagate	finally cleanup
------------	-----	------------	-------------------	-----------------

Risky Step -> try -> Exception? -> catch / propagate -> finally cleanup

Concept	Explanation
Exception	An event that disrupts normal program flow.
try/catch	Wrap risky code and handle a specific failure.
finally	Runs after try/catch for cleanup-oriented actions.
throw	Explicitly create/raise an exception.
throws	Declare that a method may pass checked exceptions to its caller.
Checked exception	Compiler requires handling or declaration.
Unchecked exception	Runtime exception; usually programming or validation defects.

Basic try-catch-finally

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Cleanup completed");
        }
    }
}
```

Custom exception for test data

```
class InvalidTestDataException extends RuntimeException {
    InvalidTestDataException(String message) {
        super(message);
    }
}

class TestDataValidator {
    static void validateUser(String user) {
        if (user == null || user.isBlank()) {
            throw new InvalidTestDataException("User is missing");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        try {
            TestDataValidator.validateUser("");
        }
    }
}
```

```

    } catch (InvalidTestDataException e) {
        System.out.println(e.getMessage());
    }
}

```

Retry-style exception handling

```

class FlakyService {
    static int attempt = 0;

    static void call() {
        attempt++;
        if (attempt < 3) {
            throw new RuntimeException("Temporary failure");
        }
        System.out.println("Success on attempt " + attempt);
    }
}

public class Main {
    public static void main(String[] args) {
        int maxRetries = 3;
        for (int i = 1; i <= maxRetries; i++) {
            try {
                FlakyService.call();
                break;
            } catch (RuntimeException e) {
                System.out.println("Attempt " + i + " failed");
                if (i == maxRetries) throw e;
            }
        }
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create InvalidLoginException, InvalidPaymentException and TestDataException. Use the right exception in each validation path.
2. Write a method that catches FileNotFoundException separately from IOException and logs different messages.
3. Build a retry utility that retries only transient failures and immediately fails for invalid input.
4. Write a bad catch block that uses catch(Exception e) and improve it using specific exception handling.

SCENARIO-BASED INTERVIEW PRACTICE

1. A test fails with TimeoutException. Should you blindly retry it? Explain what evidence you would inspect first.
2. You see empty catch blocks in a framework. What risks do they create for reporting and defect detection?
3. When would you throw a custom exception instead of returning false or null?

INTERVIEW QUICK REVISION

- Catch specific exceptions before broader ones.
- finally is for cleanup, not for business logic.
- Do not swallow exceptions silently.
- Retry should target transient failures, not every exception.
- Custom exceptions improve meaning at framework boundaries.

DAY 9 - File Handling

Ashwini K | Goal: Read, write, append, copy, move and validate files using modern Java I/O patterns suitable for test frameworks.

CORE IDEA

Read, write, append, copy, move and validate files using modern Java I/O patterns suitable for test frameworks.

Path	Validate	Open	Read / Write	Close / Verify
------	----------	------	--------------	----------------

Path -> Validate -> Open -> Read / Write -> Close / Verify

Concept	Explanation
Path	Represents a file or directory path.
Files	Utility class for reading, writing, copying, moving and checking files.
Reader / Writer	Character-oriented file I/O.
InputStream / OutputStream	Byte-oriented I/O.
try-with-resources	Automatically closes AutoCloseable resources.
File validation	Check exists, readable, size, extension and content before use.

Write and read a text file

```
import java.nio.file.*;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {
        Path path = Paths.get("result.txt");
        Files.writeString(path, "LOGIN=PASS\nSEARCH=PASS\n");
        String content = Files.readString(path);
        System.out.println(content);
    }
}
```

Count lines and passed tests

```
import java.nio.file.*;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {
        Path path = Paths.get("results.txt");
        long lines = Files.lines(path).count();
        long passed = Files.lines(path)
            .filter(line -> line.endsWith("=PASS"))
            .count();
        System.out.println("Total=" + lines);
        System.out.println("Passed=" + passed);
    }
}
```

Copy artifact after execution

```
import java.nio.file.*;
import java.io.IOException;
```

```

public class Main {
    public static void main(String[] args) throws IOException {
        Path source = Paths.get("screenshots/login.png");
        Path target = Paths.get("artifacts/login-latest.png");
        Files.createDirectories(target.getParent());
        Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);
        System.out.println("Copied: " + Files.exists(target));
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create a utility to append test execution lines in the format testId,status,timestamp.
2. Write a program that scans a folder and prints only .png screenshot files larger than 50 KB.
3. Build a log validator that fails when it finds ERROR but ignores WARN.
4. Copy failed-test screenshots into a dated archive folder and verify the copy.

SCENARIO-BASED INTERVIEW PRACTICE

1. A CI agent runs tests in a clean workspace. How should your file utility handle missing directories?
2. Why is try-with-resources safer than manually closing streams in framework code?
3. Your log file is 500 MB. Would you use readAllLines? What would you change?

INTERVIEW QUICK REVISION

- java.nio.file.Path and Files are the modern core APIs.
- Use try-with-resources for closeable resources.
- Validate paths before reading or writing.
- Stream large files instead of loading everything into memory.
- Keep artifact paths configurable, not hardcoded.

DAY 10 - Collections Fundamentals + List

Ashwini K | Goal: Understand the Collection hierarchy and choose List implementations based on access and insertion needs.

CORE IDEA

Understand the Collection hierarchy and choose List implementations based on access and insertion needs.

Data	Collection type	Implementation	Operations	Result
------	-----------------	----------------	------------	--------

Data -> Collection type -> Implementation -> Operations -> Result

Concept	Explanation
Collection	Root interface for groups of objects; List, Set and Queue are major branches.
Generics	Provide compile-time type safety, e.g. List<String>.
List	Ordered collection that permits duplicates and index-based access.
ArrayList	Fast random access; common default List choice.
LinkedList	Efficient insert/remove at known node positions; poor random access.
Autoboxing	Primitive wrapper conversion, e.g. int -> Integer.

ArrayList for test names

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> tests = new ArrayList<>();
        tests.add("Login");
        tests.add("Search");
        tests.add("Payment");
        for (String test : tests) {
            System.out.println(test);
        }
    }
}
```

Remove duplicates while keeping order

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> tests = Arrays.asList("Login", "Search", "Login", "Payment");
        List<String> unique = new ArrayList<>(new LinkedHashSet<>(tests));
        System.out.println(unique);
    }
}
```

Failure count

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
```

```
List<Boolean> results = Arrays.asList(true, false, true, false, false);
int failures = 0;
for (boolean result : results) {
    if (!result) failures++;
}
System.out.println("Failures=" + failures);
}
```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create a List of 20 test names and print only tests containing the word Login.
2. Insert a high-priority test at index 0 and explain the cost of repeated inserts at the beginning of an ArrayList.
3. Convert an array to List and sort it without modifying the source array.
4. Compare ArrayList and LinkedList for a suite that frequently reads by index but rarely inserts in the middle.

SCENARIO-BASED INTERVIEW PRACTICE

1. Your suite stores 100,000 test results and repeatedly calls get(index). Which List implementation would you start with and why?
2. Why does List allow duplicates while Set does not?
3. What is the risk of using a raw List without generics in a large automation framework?

INTERVIEW QUICK REVISION

- List preserves insertion order and supports duplicates.
- ArrayList is usually the default List choice.
- Use generics for type safety.
- LinkedList is not automatically faster; access pattern matters.
- Choose collection implementation from workload, not habit.

DAY 11 - Collections - LinkedList, Stack, Queue and Deque

Ashwini K | Goal: Model FIFO/LIFO and workflow processing using the correct collection type.

CORE IDEA

Model FIFO/LIFO and workflow processing using the correct collection type.

Input Jobs	Queue	Process	Result
------------	-------	---------	--------

Input Jobs -> Queue -> Process -> Result

Concept	Explanation
Queue	FIFO processing; first inserted item is normally processed first.
Deque	Double-ended queue; add/remove from both ends.
Stack	LIFO concept; in modern Java prefer Deque for stack behavior.
PriorityQueue	Elements are removed according to priority ordering.
offer/poll/peek	Queue-friendly operations that avoid exception-heavy APIs.

Test execution queue

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<String> queue = new ArrayDeque<>();
        queue.offer("Login");
        queue.offer("Search");
        queue.offer("Payment");

        while (!queue.isEmpty()) {
            System.out.println("Run: " + queue.poll());
        }
    }
}
```

Undo style stack with Deque

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Deque<String> history = new ArrayDeque<>();
        history.push("Open Home");
        history.push("Open Search");
        history.push("Open Booking");

        System.out.println("Current=" + history.peek());
        System.out.println("Undo=" + history.pop());
        System.out.println("Now=" + history.peek());
    }
}
```

PriorityQueue for critical tests

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> priorities = new PriorityQueue<>();
        priorities.offer(3);
        priorities.offer(1);
        priorities.offer(2);
        while (!priorities.isEmpty()) {
            System.out.println(priorities.poll());
        }
    }
}
```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Build a queue that stores test IDs and processes them in FIFO order.
2. Use Deque to simulate browser navigation history and implement back behavior.
3. Use PriorityQueue to process severity 1 defects before severity 3 defects.
4. Explain when a Queue is better than a List for work scheduling.

SCENARIO-BASED INTERVIEW PRACTICE

1. Your test runner receives work faster than workers can process it. Which collection model fits this requirement?
2. Why is Deque generally preferred over the legacy Stack class for stack behavior?
3. What does PriorityQueue guarantee and what does it not guarantee about iteration order?

INTERVIEW QUICK REVISION

- Queue = FIFO.
- Stack behavior = LIFO; use Deque.
- poll removes and returns; peek reads without removal.
- PriorityQueue orders removal by priority, not arbitrary iteration.
- Use collection interfaces in variable declarations where practical.

DAY 12 - Collections - Set and Uniqueness

Ashwini K | Goal: Remove duplicates, model unique entities and understand HashSet, LinkedHashSet and TreeSet.

CORE IDEA

Remove duplicates, model unique entities and understand HashSet, LinkedHashSet and TreeSet.

Input values	equals/hashCode	Unique set	Optional sort/order
--------------	-----------------	------------	---------------------

Input values -> equals/hashCode -> Unique set -> Optional sort/order

Concept	Explanation
Set	Collection that does not permit duplicate elements according to equality rules.
HashSet	Fast average lookup; no guaranteed iteration order.
LinkedHashSet	Maintains insertion order.
TreeSet	Stores unique elements in sorted order.
hashCode/equals	Core contract used by hash-based collections.

Unique test IDs

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Set<String> ids = new HashSet<>();
        ids.add("TC101");
        ids.add("TC102");
        ids.add("TC101");
        System.out.println(ids);
    }
}
```

Stable unique order

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Set<String> envs = new LinkedHashSet<>();
        envs.add("QA");
        envs.add("UAT");
        envs.add("QA");
        envs.add("PROD");
        System.out.println(envs);
    }
}
```

Sorted unique priorities

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Set<Integer> severities = new TreeSet<>();
    }
}
```

```
        severities.add(3);  
        severities.add(1);  
        severities.add(2);  
        severities.add(1);  
        System.out.println(severities);  
    }  
}
```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Given 50 duplicate test IDs, produce a unique collection and preserve the first-seen order.
2. Create a custom class TestKey and store it in HashSet. Implement equals and hashCode correctly.
3. Use TreeSet to print defect severities in ascending order and explain the tradeoff versus HashSet.

SCENARIO-BASED INTERVIEW PRACTICE

1. Two TestCase objects contain the same ID but are different objects. When should a Set treat them as duplicates?
2. Users complain that HashSet iteration order changes. Is that a defect? What collection would you use when order matters?
3. What bugs can occur when equals() and hashCode() disagree?

INTERVIEW QUICK REVISION

- Set removes duplicates based on equality semantics.
- HashSet: no order guarantee.
- LinkedHashSet: insertion order.
- TreeSet: sorted order and uniqueness.
- For hash collections, equals/hashCode must obey their contract.

DAY 13 - Collections - Map

Ashwini K | Goal: Store key-value data efficiently for configuration, lookup, aggregation and test-result reporting.

CORE IDEA

Store key-value data efficiently for configuration, lookup, aggregation and test-result reporting.

Key	Lookup	Value	Update / Aggregate
-----	--------	-------	--------------------

Key -> Lookup -> Value -> Update / Aggregate

Concept	Explanation
Map	Associates a key with a value; keys are unique.
HashMap	Fast average lookup; no iteration order guarantee.
LinkedHashMap	Maintains insertion order.
TreeMap	Maintains keys in sorted order.
getOrDefault	Convenient lookup with fallback.
merge	Useful for aggregation without verbose containsKey code.

Test status map

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Map<String, String> status = new HashMap<>();
        status.put("TC101", "PASS");
        status.put("TC102", "FAIL");
        System.out.println(status.get("TC102"));
    }
}
```

Failure count with merge

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> results = Arrays.asList("LOGIN", "LOGIN", "SEARCH", "LOGIN", "SEARCH");
        Map<String, Integer> count = new HashMap<>();
        for (String name : results) {
            count.merge(name, 1, Integer::sum);
        }
        System.out.println(count);
    }
}
```

Configuration lookup

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Map<String, String> config = new HashMap<>();
        config.put("browser", "chrome");
    }
}
```

```
config.put("env", "qa");  
config.put("headless", "true");  
  
String timeout = config.getDefault("timeout", "30");  
System.out.println("Timeout=" + timeout);  
}  
}
```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Create a Map<environment, baseUrl> and print the URL for a selected environment.
2. Count occurrences of each HTTP status code from a List<Integer>.
3. Group test names by module using Map<String, List<String>>.
4. Use LinkedHashMap to generate a stable execution summary in insertion order.

SCENARIO-BASED INTERVIEW PRACTICE

1. You need O(1)-average lookup by test ID across 100,000 records. Why is Map suitable?
2. What happens when you put the same key twice in a HashMap?
3. When would LinkedHashMap or TreeMap be preferable to HashMap?

INTERVIEW QUICK REVISION

- Map keys are unique.
- HashMap gives average constant-time lookup under normal hashing assumptions.
- getOrDefault and merge simplify common patterns.
- LinkedHashMap preserves insertion order.
- TreeMap sorts keys.

DAY 14 - Collections - Iterator, Comparable and Comparator

Ashwini K | Goal: Traverse collections safely and define sorting rules without tightly coupling data models.

CORE IDEA

Traverse collections safely and define sorting rules without tightly coupling data models.

Collection	Iterator / Sort	Comparison rule	Ordered / Filtered result
------------	-----------------	-----------------	---------------------------

Collection -> Iterator / Sort -> Comparison rule -> Ordered / Filtered result

Concept	Explanation
Iterator	Standard traversal abstraction with hasNext(), next() and optional remove().
ListIterator	Bidirectional iterator for Lists with richer modification/navigation support.
Comparable	Natural ordering defined inside the class using compareTo().
Comparator	External/custom ordering using compare().
Collections.sort	Sorts a List using natural order or a Comparator.

Safe removal with Iterator

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> tests = new ArrayList<>(Arrays.asList("PASS", "FAIL", "PASS", "FAIL"));
        Iterator<String> it = tests.iterator();
        while (it.hasNext()) {
            if (it.next().equals("FAIL")) {
                it.remove();
            }
        }
        System.out.println(tests);
    }
}
```

Comparator for test priority

```
import java.util.*;

class TestCase {
    String name;
    int priority;

    TestCase(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }
}

public class Main {
    public static void main(String[] args) {
        List<TestCase> tests = new ArrayList<>();
        tests.add(new TestCase("Search", 2));
    }
}
```

```

        tests.add(new TestCase("Login", 1));

        tests.sort(Comparator.comparingInt(t -> t.priority));
        for (TestCase t : tests) System.out.println(t.name);
    }
}

```

Comparable natural order

```

import java.util.*;

class Build implements Comparable<Build> {
    int number;
    Build(int number) { this.number = number; }

    public int compareTo(Build other) {
        return Integer.compare(this.number, other.number);
    }
}

public class Main {
    public static void main(String[] args) {
        List<Build> builds = Arrays.asList(new Build(12), new Build(5), new Build(8));
        Collections.sort(builds);
        builds.forEach(b -> System.out.println(b.number));
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Remove all skipped test cases from a List using Iterator without ConcurrentModificationException.
2. Sort test cases by priority, then name, using Comparator chaining.
3. Create a Release class implementing Comparable by version number.
4. Explain with code when you would choose Comparable versus Comparator.

SCENARIO-BASED INTERVIEW PRACTICE

1. Why can removing an item in a for-each loop cause ConcurrentModificationException?
2. A product owner changes sorting requirements every sprint. Would you put that logic in Comparable or Comparator?
3. Why is Comparator.comparing... often easier to read than manual compare methods?

INTERVIEW QUICK REVISION

- Iterator is the safe removal mechanism during iterator traversal.
- Comparable defines natural order inside the class.
- Comparator defines one or more external sort strategies.
- Comparator chaining supports multi-level sorting.
- Avoid modifying a collection directly during for-each traversal.

DAY 15 - Collections - Advanced Practice and Interview Mastery

Ashwini K | Goal: Combine List, Set and Map patterns in real QA/SDET utilities and solve senior-level collection problems.

CORE IDEA

Combine List, Set and Map patterns in real QA/SDET utilities and solve senior-level collection problems.

Requirement	Choose Collection	Populate	Transform	Validate	Report
-------------	-------------------	----------	-----------	----------	--------

Requirement -> Choose Collection -> Populate -> Transform -> Validate -> Report

Concept	Explanation
Choose by access pattern	List for ordered sequences, Set for uniqueness, Map for keyed lookup.
Immutable views / factories	Use List.of(), Set.of(), Map.of() when fixed data is intended.
Defensive copying	Copy incoming collections when object ownership matters.
Nested collections	Map<String, List<String>> and Map<String, Set<String>> model grouped data.
Performance mindset	Consider lookup, insertion, ordering, duplication, memory and concurrency requirements.
Framework integration	Collections commonly back test data, suites, result aggregation and configuration.

Group failures by module

```
import java.util.*;

public class Main {
    static class Failure {
        String module, test;
        Failure(String module, String test) { this.module = module; this.test = test; }
    }

    public static void main(String[] args) {
        List<Failure> failures = Arrays.asList(
            new Failure("Login", "InvalidPassword"),
            new Failure("Search", "NoResults"),
            new Failure("Login", "LockedUser")
        );

        Map<String, List<String>> grouped = new LinkedHashMap<>();
        for (Failure f : failures) {
            grouped.computeIfAbsent(f.module, k -> new ArrayList<>()).add(f.test);
        }
        System.out.println(grouped);
    }
}
```

Find first duplicate

```
import java.util.*;

public class Main {
```

```

static String firstDuplicate(List<String> values) {
    Set<String> seen = new HashSet<>();
    for (String value : values) {
        if (!seen.add(value)) return value;
    }
    return null;
}

public static void main(String[] args) {
    System.out.println(firstDuplicate(Arrays.asList("A", "B", "C", "B", "D")));
}

```

Top failing tests

```

import java.util.*;

class TestMetric {
    String name;
    int failures;

    TestMetric(String name, int failures) {
        this.name = name;
        this.failures = failures;
    }
}

public class Main {
    public static void main(String[] args) {
        List<TestMetric> metrics = new ArrayList<>(Arrays.asList(
            new TestMetric("Payment", 8),
            new TestMetric("Login", 3),
            new TestMetric("Search", 5)
        ));
        metrics.sort(Comparator.comparingInt((TestMetric t) -> t.failures).reversed());
        metrics.forEach(t -> System.out.println(t.name + " -> " + t.failures));
    }
}

```

Collection-based test data validation

```

import java.util.*;

public class Main {
    static boolean isValid(Map<String, String> data) {
        Set<String> required = Set.of("username", "password", "env");
        return data.keySet().containsAll(required);
    }

    public static void main(String[] args) {
        Map<String, String> data = Map.of(
            "username", "ashwini",
            "password", "secret",
            "env", "qa"
        );
        System.out.println(isValid(data));
    }
}

```

HANDS-ON ASSIGNMENTS - 5+ YEARS PRACTICE

1. Build a TestSuite object containing List<TestCase>, Set<String> tags and Map<String,String> metadata. Choose sensible collection implementations.
2. Given a list of 10,000 results, produce counts by status and module in one pass where practical.
3. Find the top 3 failing tests using sorting or a priority-based approach. Explain the tradeoff.
4. Build a de-duplicated execution plan that keeps test order and groups by module.

5. Create a mini in-memory defect dashboard using Map, Set and List together.

SCENARIO-BASED INTERVIEW PRACTICE

1. A senior engineer asks why your field is List when only unique IDs are needed. How would you justify or change it?
2. Your result aggregation becomes slow after data grows from 1,000 to 1,000,000 records. Which collection decisions would you inspect first?
3. Your framework requires stable iteration order for reproducible reports. Which collection choices would you evaluate?
4. You must support multiple sorting strategies for the same TestCase model. Explain the Comparator design.

INTERVIEW QUICK REVISION

- List -> ordered sequence; Set -> uniqueness; Map -> key/value lookup.
- Hash vs LinkedHashMap vs Tree usually means performance vs order vs sorting tradeoffs.
- Use nested collections to model grouped test data.
- At senior level, explain collection choice from requirements and access patterns.
- Interview answers should cover correctness, readability and performance considerations.

DAY 5-15 FINAL REVISION CHECKLIST

Ashwini K | Classroom + Interview Readiness

Area	You should be able to
Classes & Objects	Create classes, constructors, objects, instance data and reusable behavior.
OOP	Explain and code encapsulation, inheritance, polymorphism, abstraction and interfaces.
Exceptions	Choose specific exception handling, custom exceptions, propagation and retry rules.
Files	Read/write/append/scan/copy files and validate artifacts safely.
Collections	Choose List/Set/Map/Queue types from the actual requirement.
Senior reasoning	Explain tradeoffs: order, duplicates, lookup, memory, sorting and maintainability.

FINAL INTERVIEW CHALLENGE

Design an in-memory QA execution service. It should store test metadata, prevent duplicate test IDs, process selected tests in a queue, aggregate results by module, handle transient execution failures, write a summary file, and expose multiple report strategies. Explain where you would use class/object, inheritance, interface, exception handling, file handling and each collection type.

ASHWINI K TRAINER NOTE

Do not memorize the programs. First predict the output, then run them, then modify the input and explain what changes. Senior-level confidence comes from understanding why a design works.